Small Project 12/03/2026

Machine Learning Approaches for Network Intrusion Detection: A Comparative Analysis Across Multiple Datasets

Focchi Fabio 12542524

1 Introduction

The increasing volume and sophistication of cyberattacks have made Intrusion Detection Systems (IDS) a cornerstone of modern network security. Traditional rule-based detectors rely on manually crafted signatures and are brittle against zero-day exploits and rapidly evolving attack vectors [4]. Machine Learning (ML) offers a compelling alternative by learning statistical patterns directly from labelled or unlabelled traffic data, enabling generalization to novel threats without explicit rule engineering.

This project is motivated by three complementary observations. **Methodological diversity:** supervised approaches span a wide spectrum from linear probabilistic models (Logistic Regression) to kernel-based classifiers (SVM), tree ensembles (Random Forest), and deep architectures (ANN), each embodying different inductive biases and computational trade-offs. **Dataset heterogeneity:** real-world IDS deployments differ in network topology, traffic mix, and attack tooling; understanding how model performance varies across datasets with different statistical properties is essential for building robust detectors. **Complementarity of learning paradigms:** supervised learning requires labels, which are expensive and may become stale; unsupervised clustering can reveal latent structure independently of labels, shedding light on the intrinsic separability of normal and malicious traffic.

The study covers five objectives: (i) dataset-specific preprocessing pipelines tailored to each dataset's characteristics; (ii) training and evaluation of six supervised classifiers using a consistent evaluation protocol; (iii) convergence analysis to assess how performance evolves with training-set size; (iv) K-Means clustering with PCA visualization and validity metrics; and (v) systematic cross-dataset comparison to extract insights transferable to practitioners.

The two datasets selected represent two distinct generations of IDS benchmarks. The KDD Cup 1999 dataset, despite its age, remains a widely used baseline due to its controlled simulation environment and near-perfect class separability. UNSW-NB15 represents a more realistic and challenging scenario, featuring modern attack tooling, higher feature dimensionality, and inherent class overlap across nine attack families. Evaluating the same methodology across both datasets allows conclusions to be anchored to both a well-understood baseline and a more practically relevant setting, enabling a more robust assessment of each model's generalization capacity.

2 Theoretical Background

2.1 Preprocessing Foundations

Before any model is trained, raw features must be transformed into a numerically suitable representation. Two operations are central to this project.

Standardization (Z-score scaling) maps each feature j to zero mean and unit variance:

$$\tilde{x}_{ij} = \frac{x_{ij} - \hat{\mu}_j}{\hat{\sigma}_j}, \tag{1}$$

where $\hat{\mu}_j$ and $\hat{\sigma}_j$ are estimated on the training set only to prevent data leakage. Standardization is necessary for distance-based models (KNN, SVM) and gradient-based optimizers (LR, ANN), whose loss landscapes are sensitive to feature scale.

Categorical encoding converts symbolic features into numeric form. Label encoding assigns an integer code $\tilde{x}_{ij} = \text{code}(x_{ij}) \in \{0, \ldots, C_j - 1\}$ and is appropriate when the model can exploit ordinal structure natively, as tree-based classifiers do. One-hot encoding instead creates a binary indicator column for each category value, imposing no spurious ordering; it is required for linear and distance-based models where arbitrary integer codes would mislead the optimizer.

Winsorization clips heavy-tailed distributions at the q_α and $q_{1-\alpha}$ quantiles:

$$\tilde{x}_{ij} = \max\big(q_\alpha^{(j)},\, \min(q_{1-\alpha}^{(j)},\, x_{ij})\big), \tag{2}$$

with $\alpha = 0.01$ in this project. This eliminates pathological extremes that would otherwise dominate Euclidean distances and distort the scaler estimates.

2.2 Supervised Learning

Supervised learning seeks a hypothesis $f_{\hat{\theta}}$ from a labelled training set $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ by minimizing the regularized empirical risk [4]:

$$\hat{\theta} = \arg\min_\theta \underbrace{\frac{1}{N}\sum_{i=1}^{N} \mathcal{L}(f(\mathbf{x}_i;\theta), y_i)}_{\text{empirical risk}} + \underbrace{\lambda\,\Omega(\theta)}_{\text{regularizer}}, \tag{3}$$

where $\lambda > 0$ and $\Omega(\theta)$ penalizes model complexity (e.g. $\|\theta\|_2^2$ for Ridge, $\|\theta\|_1$ for Lasso). In binary classification $\mathcal{Y} = \{0, 1\}$; predictions are made via a threshold τ: $\hat{y} = \mathbf{1}[P(y{=}1|\mathbf{x}) \geq \tau]$.

2.2.1 Logistic Regression

Logistic Regression is a discriminative parametric linear classifier that models the class-posterior through the sigmoid function:

$$P(y{=}1|\mathbf{x};\boldsymbol{\beta}) = \sigma(\boldsymbol{\beta}^\top \mathbf{x} + \beta_0) = \frac{1}{1 + e^{-(\boldsymbol{\beta}^\top \mathbf{x} + \beta_0)}}. \tag{4}$$

Parameters are estimated by minimizing the binary cross-entropy:

$$\mathcal{L}(\boldsymbol{\beta}) = -\frac{1}{N}\sum_{i=1}^{N}\big[y_i \log \sigma(\mathbf{x}_i^\top \boldsymbol{\beta}) + (1-y_i)\log(1-\sigma(\mathbf{x}_i^\top \boldsymbol{\beta}))\big], \tag{5}$$

with ℓ_2 regularization added as $\lambda\|\boldsymbol{\beta}\|_2^2$. In practice the regularization strength is controlled via the inverse parameter $C = 1/\lambda$: smaller C implies stronger regularization. The linear decision boundary serves as a strong interpretable baseline but limits capacity on non-linearly separable data.

2.2.2 K-Nearest Neighbours (KNN)

KNN assigns the plurality class among the k closest training points:

$$\hat{y}(\mathbf{x}) = \arg\max_{c\in\mathcal{Y}} \sum_{\mathbf{x}_j\in\mathcal{N}_k(\mathbf{x})} \mathbf{1}[y_j = c], \quad \mathcal{N}_k(\mathbf{x}) = k \text{ nearest under } \|\cdot\|_2. \tag{6}$$

Inference complexity is $\mathcal{O}(Nd)$ per query, making it impractical at large scale; PCA dimensionality reduction is applied for Dataset 2 to make the algorithm computationally feasible.

2.2.3 Support Vector Machine (SVM)

An SVM seeks the hyperplane maximizing the geometric margin $2/\|\mathbf{w}\|$. The soft-margin formulation:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \ \tfrac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i,\ \xi_i \geq 0, \tag{7}$$

where $C > 0$ balances margin and misclassification penalty. The *kernel trick* replaces inner products with $K(\mathbf{x}_i, \mathbf{x}_j)$; the RBF kernel $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma\|\mathbf{x}_i - \mathbf{x}_j\|^2)$ enables non-linear boundaries but scales between $\mathcal{O}(N^2)$ and $\mathcal{O}(N^3)$, motivating the switch to LinearSVC on the larger Dataset 2.

2.2.4 Decision Tree

A Decision Tree recursively selects splits (j^*, t^*) maximizing Gini impurity reduction:

$$(j^*, t^*) = \arg\max_{j,t}\Big[G(\mathcal{S}) - \tfrac{|\mathcal{S}_L|}{|\mathcal{S}|}G(\mathcal{S}_L) - \tfrac{|\mathcal{S}_R|}{|\mathcal{S}|}G(\mathcal{S}_R)\Big], \qquad G(\mathcal{S}) = 1 - \sum_k \hat{p}_k^2. \tag{8}$$

Depth capping and minimum-leaf-size thresholds prevent overfitting.

2.2.5 Random Forest

Random Forest [2] is an ensemble of T decision trees, each trained on a bootstrap replicate; at each split only $m \ll d$ random features are eligible, decorrelating the trees:

$$\hat{y}(\mathbf{x}) = \arg\max_{c} \frac{1}{T} \sum_{b=1}^{T} \mathbf{1}[\hat{y}_b(\mathbf{x}) = c]. \tag{9}$$

The ensemble variance is proportional to the average pairwise correlation ρ between trees [2]: $\mathrm{Var}(\bar{f}) \approx \rho\, \overline{\mathrm{Var}(t_b)}$, so low ρ (achieved by random feature subsets) is key to variance reduction.

2.2.6 Artificial Neural Network (ANN)

A fully-connected MLP with L hidden layers applies a cascade of affine transformations and non-linear activations:

$$\mathbf{a}^{(0)} = \mathbf{x}, \qquad \mathbf{a}^{(l)} = \phi\big(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}\big), \quad l = 1, \ldots, L, \tag{10}$$

where $\phi(\cdot) = \max(0, \cdot)$ denotes the ReLU activation on hidden layers; the output neuron applies a sigmoid $\sigma(\cdot)$ to produce a probability in $[0, 1]$. The network is trained by minimizing binary cross-entropy via the Adam optimizer; Dropout and Batch Normalization regularize training and stabilize convergence.

2.3 Evaluation Metrics for Supervised Learning

In binary classification, every prediction falls into one of four categories summarised by the *confusion matrix*:

Table 1: Confusion matrix for binary IDS classification.

		Predicted	
		Normal (0)	Attack (1)
Actual	Normal (0)	True Negative (TN)	False Positive (FP)
	Attack (1)	False Negative (FN)	True Positive (TP)

TP (True Positive): an attack correctly identified as attack. **TN** (True Negative): normal traffic correctly identified as normal. **FP** (False Positive): normal traffic wrongly flagged as attack (false alarm). **FN** (False Negative): an attack wrongly classified as normal (missed detection).

From these counts the standard metrics are derived:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}, \qquad \text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \tag{11}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \qquad F_1 = \frac{2\,\text{TP}}{2\,\text{TP} + \text{FP} + \text{FN}}. \tag{12}$$

In IDS contexts, low precision implies a high false-alarm rate burdening analysts, while low recall means attacks go undetected, a severe security failure.

The *Receiver Operating Characteristic* (ROC) curve plots the True Positive Rate $\text{TPR} = \text{TP}/(\text{TP} + \text{FN})$ against the False Positive Rate $\text{FPR} = \text{FP}/(\text{FP} + \text{TN})$ as the classification threshold τ varies from 0 to 1. A model with no discriminative power follows the diagonal ($\text{AUC} = 0.5$); a perfect model reaches the top-left corner ($\text{TPR} = 1$, $\text{FPR} = 0$, $\text{AUC} = 1$). The *Area Under the Curve* (ROC-AUC) $= \int_0^1 \text{TPR}(\text{FPR})\, d\text{FPR}$ is threshold-independent and robust to class imbalance, making it the primary ranking metric when comparing models across datasets with different class proportions. Log-Loss penalizes miscalibrated probability estimates [4].

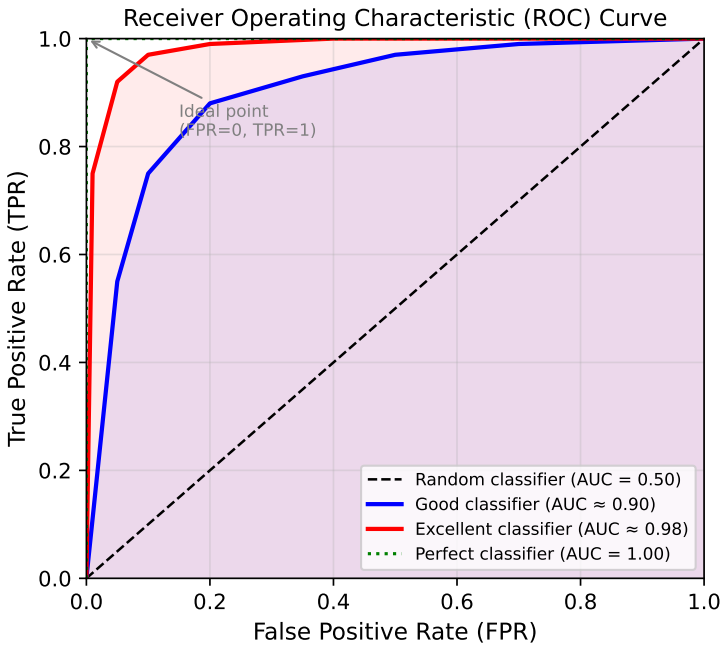

Figure 1: Illustrative ROC curves for three classifier types. The dashed diagonal represents a random classifier ($\text{AUC} = 0.50$); curves bowing toward the top-left corner indicate progressively better discrimination. The ideal classifier reaches $\text{TPR} = 1$, $\text{FPR} = 0$ ($\text{AUC} = 1.00$).

2.4 Unsupervised Learning: K-Means

K-Means minimizes the Within-Cluster Sum of Squares:

$$J(\boldsymbol{\mu}, \mathbf{R}) = \sum_{i=1}^{N} \sum_{k=1}^{K} r_{ik} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2, \quad r_{ik} \in \{0, 1\}, \ \sum_k r_{ik} = 1, \tag{13}$$

via Lloyd's algorithm alternating between (i) assigning each point to the nearest centroid and (ii) updating each centroid to the mean of its assigned points. The algorithm converges in finite steps since J decreases monotonically and the number of distinct partitions is finite. Convergence is to a *local* minimum; multiple restarts with K-Means++ initialization [1] are used to reduce this risk. The quality of initialization has been shown to have a significant impact on clustering accuracy, particularly when clusters overlap [3].

Known limitations: K must be set a priori; the Euclidean objective assumes hyperspherical, equally-sized clusters; squared distances amplify the influence of outliers.

2.4.1 Clustering Validity Metrics

External metrics compare cluster assignments $\hat{\mathbf{c}}$ to ground-truth labels $\mathbf{y}$ over all $\binom{N}{2}$ pairs, where $\mathrm{TP}_c/\mathrm{TN}_c/\mathrm{FP}_c/\mathrm{FN}_c$ are defined analogously to the supervised case but for pairs rather than individual samples:

$$\mathrm{RI} = \frac{\mathrm{TP}_c + \mathrm{TN}_c}{\mathrm{TP}_c + \mathrm{TN}_c + \mathrm{FP}_c + \mathrm{FN}_c} \in [0, 1], \tag{14}$$

$$\mathrm{FMI} = \sqrt{\frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c} \cdot \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FN}_c}} \in [0, 1], \tag{15}$$

$$\mathrm{JI} = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c} \in [0, 1]. \tag{16}$$

FMI is more discriminative than RI for imbalanced partitions; JI ignores TN_c pairs, making it more sensitive to cluster fragmentation.

Internal metrics assess geometric quality without labels. The Silhouette Index (SI) [6] averages per-point scores:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i),\, b(i)\}} \in [-1, +1], \tag{17}$$

where $a(i)$ is the mean intra-cluster distance (cohesion) and $b(i)$ the mean distance to the nearest other cluster (separation). A value close to $+1$ indicates a well-separated, compact cluster; values near 0 indicate overlapping clusters; negative values indicate likely misassignment. The Davies–Bouldin Index (DBI) $= K^{-1} \sum_k \max_{l \neq k} (\sigma_k + \sigma_l)/d(\boldsymbol{\mu}_k, \boldsymbol{\mu}_l)$ is lower for better clusterings. The Calinski–Harabász Index (CHI) is the ratio of between- to within-cluster scatter scaled by degrees of freedom; the Dunn Index (DI) is the ratio of minimum inter-cluster distance to maximum intra-cluster diameter; both are higher for better clusterings.

3 Practical Case Studies

This study employs two benchmark datasets for network intrusion detection. Both are framed as binary classification tasks (normal vs. attack) but differ substantially in origin, feature space, and statistical properties.

3.1 Dataset Description

3.1.1 Dataset 1: Kaggle Network Intrusion (KDD Cup 1999)

The first dataset is derived from the KDD Cup 1999 benchmark [7], which captured TCP/IP traffic from a simulated U.S. Air Force network over nine weeks, deliberately subjected to a wide variety of intrusion techniques. The dataset originates from the DARPA 1998 evaluation programme and has been extensively analysed in the literature [8]. Only `Train_data.csv` is used; the train/test split is derived programmatically as described in Section 3.2.1.

Each record contains **41 input features**: 9 basic connection features (`duration`, `protocol_type`, `service`, `flag`, byte counts), 13 content features (failed logins, root-shell activations, file creations), and 19 traffic statistics (two-second window counts, SYN-error rates, same-host ratios). Three features are categorical (`protocol_type`, `service`, `flag`); the target `class` takes values `normal` and `anomaly`.

A well-known limitation of the KDD Cup 99 data is the high proportion of duplicate records: [8] found that approximately 78% of training records and 75% of test records are repeated instances. This redundancy biases classifiers towards the majority pattern and inflates reported accuracy, which partly explains why all six models achieve accuracy above 0.99 on this dataset. Results on Dataset 1 should therefore be interpreted as an upper bound rather than a realistic estimate of operational performance.

Table 2: Class distribution – Dataset 1.

Split	Normal	Anomaly	Total
Training set	8,991	7,887	16,878
Test set	4,458	3,856	8,314

3.1.2 Dataset 2: UNSW-NB15

The UNSW-NB15 dataset was generated in 2015 at UNSW Canberra using the IXIA PerfectStorm tool in a hybrid testbed combining real modern attack tools with synthetic legitimate traffic [5]. Ground-truth labels cover nine attack families: *Fuzzers*, *Analysis*, *Backdoors*, *DoS*, *Exploits*, *Generic*, *Reconnaissance*, *Shellcode*, and *Worms*. The project uses the pre-split files `UNSW_NB15_training-set.csv` and `UNSW_NB15_testing-set.csv`; the multi-class column `attack_cat` is excluded from features.

After dropping the administrative `id` column, each record contains **44 input features** spanning flow, content, time, and connection statistics. Three are categorical: `proto` ($\leq$ 133 unique values), `service` (13), `state` (11). Key challenges: high cardinality of `proto`, heavy-tailed numeric distributions, and class imbalance that differs markedly between the training and test splits.

Table 3: Class distribution – Dataset 2 (UNSW-NB15).

Split	Normal (y=0)	Attack (y=1)	Total
Training set	56,000	119,341	175,341
Test set	37,000	45,332	82,332

Table 4: Structural comparison of the two datasets.

Property	Dataset 1 (Kaggle)	Dataset 2 (UNSW-NB15)
Origin year	1999 (KDD Cup)	2015
Input features	41	44 (after drop `id`)
Categorical features	3	3
Target values	`normal/anomaly`	0/1
Outlier handling	Not required	Winsorization
Encoding strategy	Label encoding	One-Hot encoding
Cardinality reduction	Not applied	Yes (`service`: top-10)

3.2 Data Preprocessing

3.2.1 Dataset 1

The pipeline consists of three sequential steps.

Categorical encoding. The three categorical features and the target are mapped to integer codes via label encoding: $\tilde{x}_{ij} = \text{code}(x_{ij}) \in \{0, \ldots, C_j - 1\}$. The target is additionally inverted so that attack= 1, normal= 0. Label encoding is preferred over one-hot encoding because tree-based models, which dominate on this dataset, can exploit integer-coded categories natively.

Standardization. All 41 features are standardized with `StandardScaler` (Z-score) fitted on the full dataset:

$$\tilde{x}_{ij} = \frac{x_{ij} - \hat{\mu}_j}{\hat{\sigma}_j}. \tag{18}$$

This is essential for distance-based models (KNN, SVM) and gradient-based optimizers (LR, ANN).

Train/test split. The scaled data is partitioned 67%/33% with `random_state=42`, after fitting the scaler to prevent leakage. The resulting shapes are $\mathbf{X}_{\text{train}} \in \mathbb{R}^{16878\times 41}$ and $\mathbf{X}_{\text{test}} \in \mathbb{R}^{8314\times 41}$.

3.2.2 Dataset 2

The pipeline is more elaborate, reflecting UNSW-NB15's greater heterogeneity. Training and test data are **concatenated before any transformation** to ensure a consistent feature space after encoding.

Cardinality reduction. Only the 10 most frequent values of `service` are retained; all others are collapsed to `other`, reducing its one-hot columns from 13 to 11.

Outlier clipping (Winsorization). All numeric columns are clipped at the 1st–99th percentile computed on the combined dataset:

$$\tilde{x}_{ij} = \max\big(q_{0.01}^{(j)}, \min(q_{0.99}^{(j)}, x_{ij})\big). \tag{19}$$

This eliminates pathological extremes without discarding the bulk of the distribution.

One-hot encoding. The three categorical features (`proto`, `service`, `state`) are one-hot encoded with `drop_first=True`. One-hot encoding is necessary because these features have no natural ordinal structure; assigning arbitrary integer codes would impose a spurious ordering that misleads linear and distance-based models. After encoding the combined matrix has shape $257{,}673 \times 191$.

Standardization and split recovery. A `StandardScaler` is fitted on the full combined matrix and applied uniformly. The original train/test split is then recovered by slicing at row N_{train}.

Table 5: Comparison of preprocessing pipelines.

Aspect	Dataset 1	Dataset 2
Encoding	Label (3 features)	One-Hot (`proto`, `service`, `state`)
Outlier handling	None	Winsorization (1–99%)
Cardinality red.	Not applied	`service`: top-10 + `other`
Scaler scope	Full training set	Combined train+test
Split origin	Programmatic 67/33	Pre-defined by authors
Final features	41	191

4 Supervised Learning

4.1 Model Configurations

Six classifiers span the full spectrum of inductive biases relevant to binary intrusion detection. Table 6 provides an overview; Tables 7 and 8 detail the dataset-specific hyperparameters.

Table 6: Overview of supervised classifiers.

Model	Type	Parametric	Interpretable	Scalability
Logistic Regression	Linear	Yes	High	High
KNN	Non-linear	No	Low	Low
SVM	Non-linear	Yes	Low	Medium
Decision Tree	Non-linear	No	High	High
Random Forest	Non-linear	No	Medium	High
ANN (MLP)	Non-linear	Yes	Low	High

Table 7: Model configurations for Dataset 1.

Model	Configuration
Logistic Regression	5-fold GridSearch over {`liblinear`, `newton-cg`, `lbfgs`, `sag`, `saga`}, C=1, `max_iter=10000`
KNN	k swept over $\{1, \ldots, 39\}$; best k selected by test accuracy
SVM	RBF kernel, C=1, `gamma='scale'`, `probability=True`; `SelectKBest`(mutual info, k=30) before fitting
Decision Tree	Gini, `max_depth=10`, `min_samples_split=20`, `random_state=42`
Random Forest	100 trees, `max_depth=15`, `min_samples_split=10`, `random_state=42`
ANN	41→32 (ReLU)→BN→Drop(0.3)→16 (ReLU)→BN→Drop(0.3)→1 (σ) Adam lr=0.001, batch 64, max 100 epochs, val-split 20% EarlyStopping (patience 6), ReduceLROnPlateau (factor 0.5, patience 3)

Table 8: Model configurations for Dataset 2. Key differences from DS1 are driven by the larger size and higher dimensionality of UNSW-NB15.

Model	Configuration
Logistic Regression	Fixed `liblinear` (efficient on sparse OHE features), C=1, `max_iter=10000`
KNN	PCA(10 components) pre-processing + `kd_tree`, k=3, `leaf_size=30`
SVM	LinearSVC (scales $\mathcal{O}(N)$ vs. $\mathcal{O}(N^2)$ for RBF), C=1, `max_iter=5000`; Platt scaling via `CalibratedClassifierCV(cv=3)`
Decision Tree	Gini, `max_depth=None`, `min_samples_split=50`, `min_samples_leaf=10`, `class_weight='balanced'`, `random_state=42`
Random Forest	300 trees, `max_depth=25`, `min_samples_split=20`, `min_samples_leaf=5`, `class_weight='balanced'`, `random_state=42`
ANN	191→128→Drop(0.3)→64→BN→Drop(0.3)→32→BN→Drop(0.2)→1 (σ) Adam lr=0.0005, batch 128, max 60 epochs, val-split 30% EarlyStopping (patience 6), ReduceLROnPlateau (factor 0.5, patience 3)

4.2 Results

Table 9: Supervised performance on Dataset 1 (test set, 33% split). Test set results.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Log-Loss
Logistic Regression	0.9544	0.9564	0.9448	0.9506	0.9902	0.1246
KNN	0.9947	0.9971	0.9914	0.9943	0.9966	0.1222
SVM (RBF)	0.9909	0.9891	0.9912	0.9902	0.9988	0.0338
Decision Tree	0.9930	0.9925	0.9925	0.9925	0.9962	0.1394
Random Forest	0.9974	0.9990	0.9953	0.9971	0.9999	0.0144
ANN	0.9905	0.9899	0.9896	0.9898	0.9991	0.0286

Table 10: Supervised performance on Dataset 2 (pre-defined test set). Test set results.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Log-Loss
Logistic Regression	0.8402	0.8086	0.9297	0.8650	0.9565	0.2613
KNN (PCA-10)	0.8528	0.8104	0.9564	0.8774	0.9102	2.4997
SVM (Linear)	0.8088	0.7465	0.9882	0.8505	0.9427	0.3992
Decision Tree	0.8888	0.8563	0.9589	0.9047	0.9515	1.2112
Random Forest	0.9029	0.8680	0.9713	0.9168	0.9842	0.1932
ANN	0.8494	0.7954	0.9782	0.8773	0.9760	0.2455

4.3 Confusion Matrices and ROC Curves

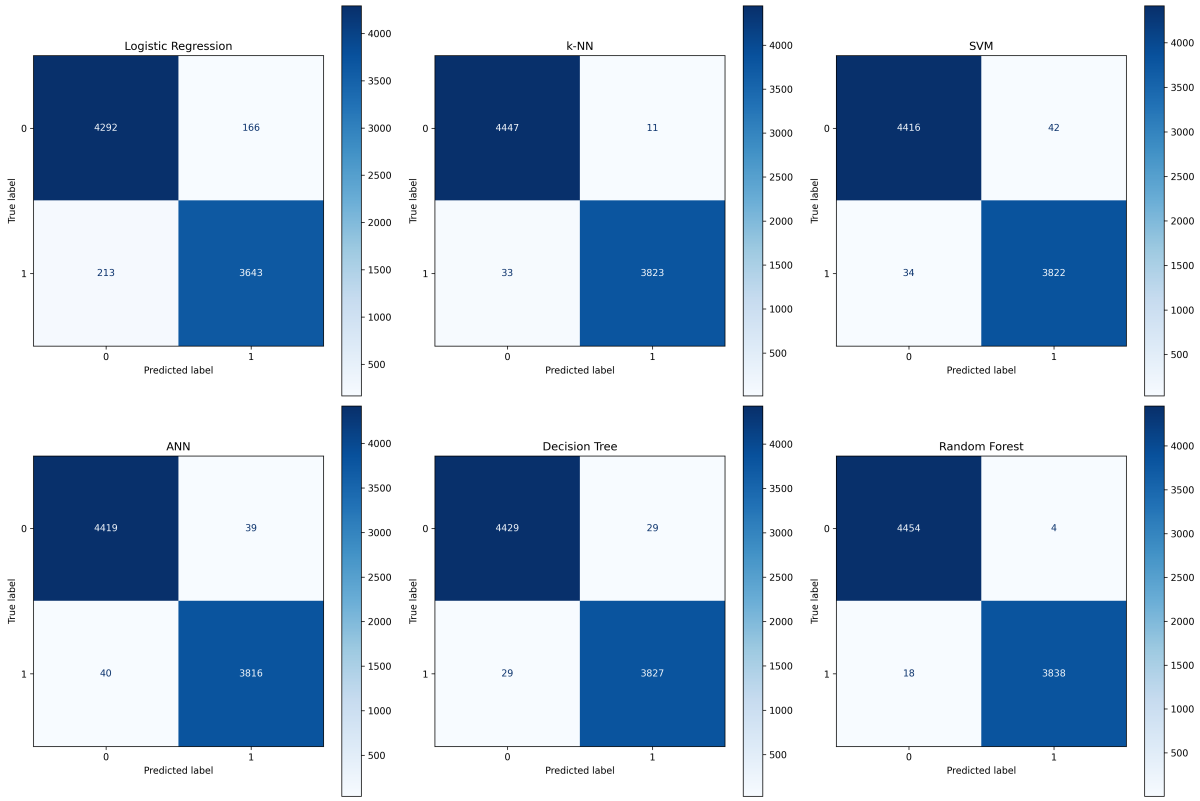

Figure 2: Confusion matrices for all six models on Dataset 1. Rows: true label; columns: predicted label. Class 0 = normal, class 1 = attack.

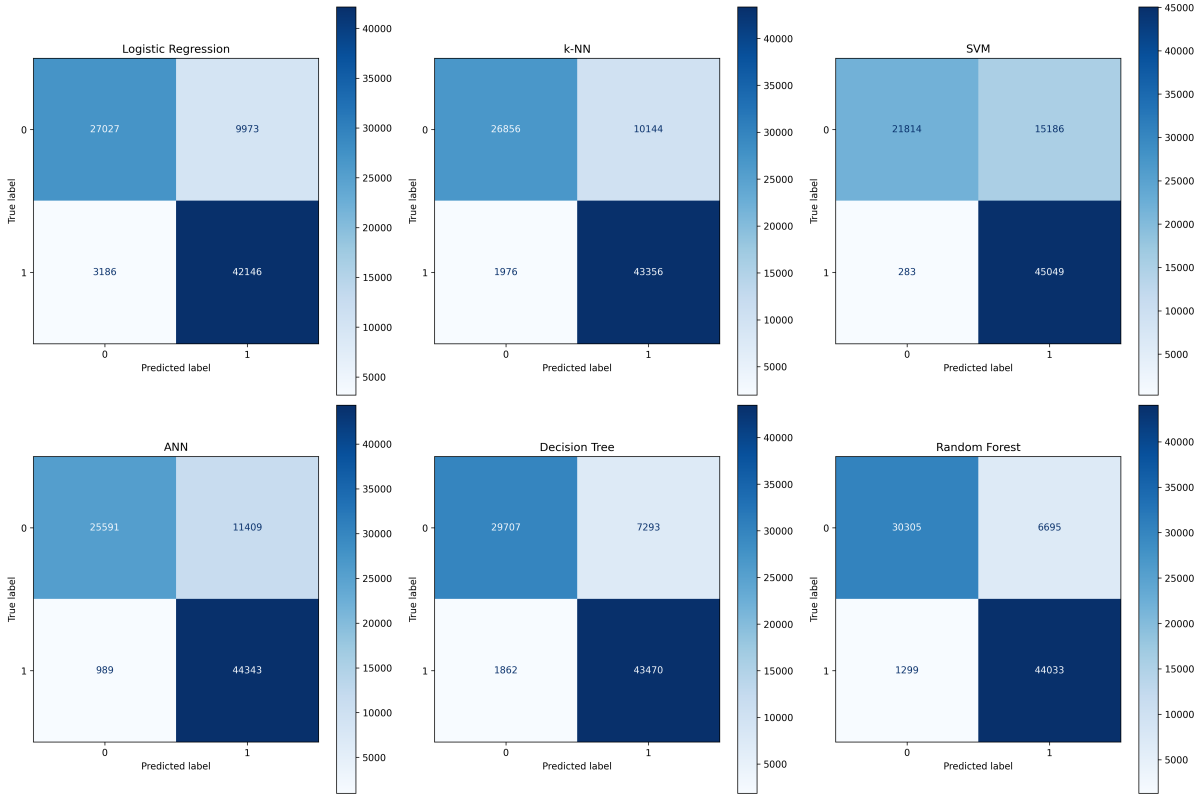

Figure 3: Confusion matrices for all six models on Dataset 2.

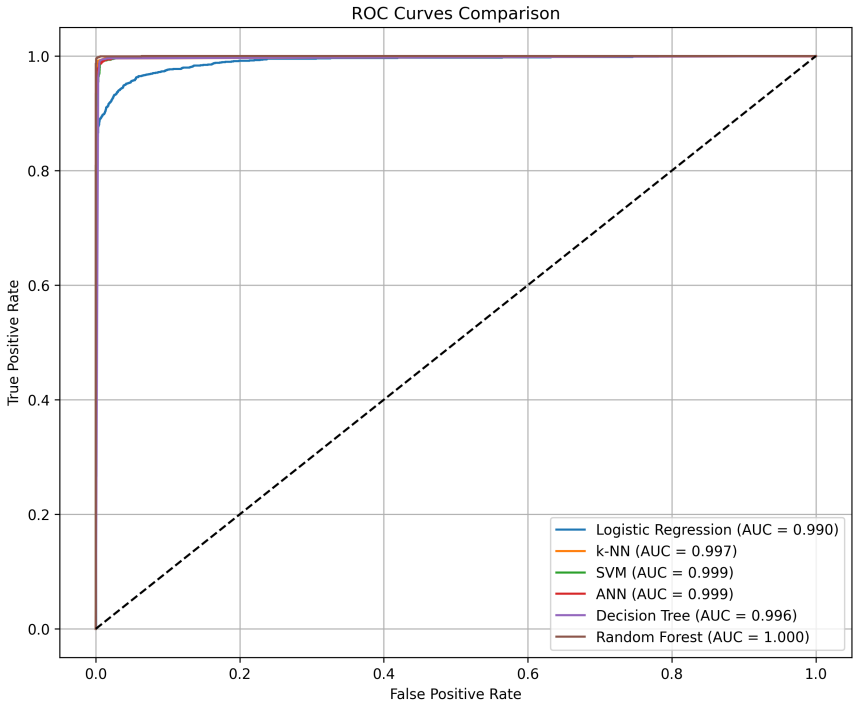

Figure 4: ROC curves for all six models on Dataset 1. Dashed diagonal: random classifier (AUC= 0.5).

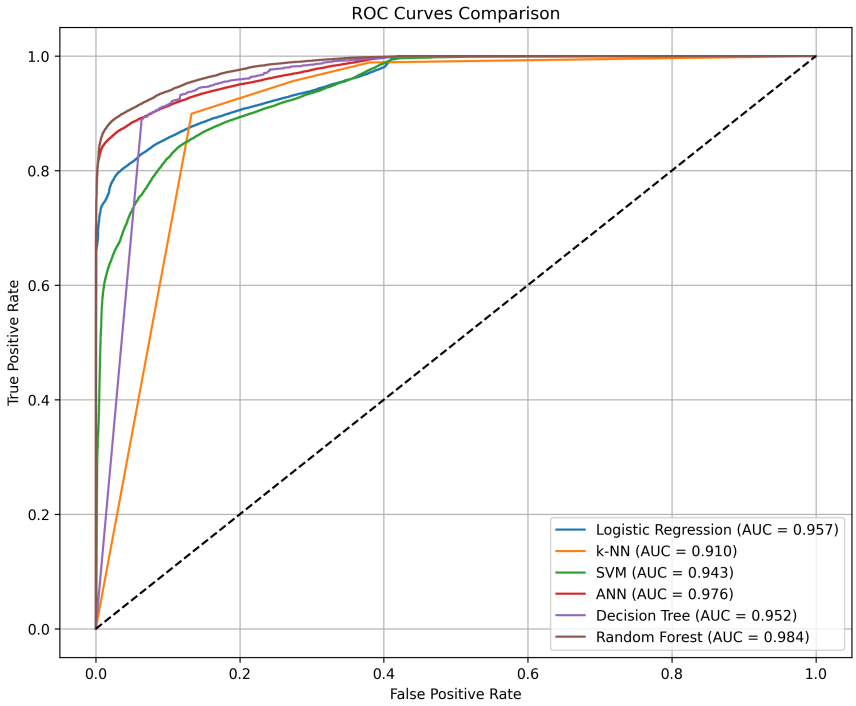

Figure 5: ROC curves for all six models on Dataset 2.

4.4 Comparative Analysis

Dataset 1. Non-linear models consistently outperform Logistic Regression, confirming that the KDD-derived feature space contains clear non-linear structure. Random Forest and Decision Tree benefit from their ability to partition the space with axis-aligned splits, well suited to the discrete and label-encoded categorical features. The RBF-SVM, operating on the 30 most informative features selected by mutual information, achieves competitive precision. The ANN, regularized by Dropout and Batch Normalization, ranks among the top models. The performance gap between Logistic Regression and the best non-linear model directly quantifies the degree of non-linearity in the task. On Dataset 1 all models achieve Recall above 0.98 for both classes, meaning the false-negative and false-positive rates are operationally negligible across the board.

Dataset 2. Random Forest achieves the best overall performance (highest accuracy, F1, and ROC-AUC), benefiting from 300 deep trees with class-weight balancing on a large heterogeneous dataset. ANN ranks second with strong generalization from its deeper and wider architecture. LinearSVC achieves very high attack recall (0.988) but at the cost of a severely depressed normal recall (0.590), meaning it would flag approximately one in two legitimate connections as malicious in production, making it operationally impractical despite its high F1. Decision Tree offers a better balance (Recall(0)=0.800, Recall(1)=0.959) but with higher Log-Loss, indicating poorly calibrated probabilities. Logistic Regression and KNN (PCA-10) show the weakest performance: the former is

limited by the non-linear nature of the task, the latter by the information loss inherent in reducing 191 features to 10 principal components. Random Forest achieves the best tradeoff with Recall(0)=0.820 and Recall(1)=0.971, making it the only model that keeps both false-alarm and missed-attack rates at operationally acceptable levels.

Cross-dataset. Absolute performance is generally higher on Dataset 1, reflecting the simpler and more controlled KDD simulation environment. The relative model ranking is only partially stable across the two datasets: Random Forest and ANN consistently rank among the top two on both benchmarks, providing the strongest evidence for their generalizability. However, the ranking of the remaining models shifts substantially. KNN ranks second on Dataset 1 (accuracy 0.9947) but drops to fifth on Dataset 2, where PCA dimensionality reduction degrades its effectiveness. Decision Tree ranks above ANN on Dataset 2 by accuracy but below it by ROC-AUC, illustrating how metric choice influences model selection conclusions. LinearSVC, competitive on Dataset 1 via the RBF kernel, ranks last on Dataset 2 where the linear boundary is insufficient for the more complex feature space. Notably, the ranking by ROC-AUC on Dataset 2 (RF > ANN > LogReg > DT > LinearSVC > KNN) differs from the ranking by accuracy (RF > DT > KNN > ANN > LogReg > LinearSVC), underscoring that the choice of evaluation metric materially affects model selection in IDS contexts where both discrimination and calibration matter.

4.5 Convergence Analysis

4.5.1 Experimental Protocol

For each dataset, six models are evaluated at 10 equally spaced training fractions $s \in \{0.10, 0.20, \ldots, 1.00\}$ of the full `X_train`. At each fraction, $n = 100$ independent stratified random subsamples are drawn; each subsample trains the model from scratch, and performance is evaluated on the *fixed* test set. Results are visualized as box plots (100 iterations per box), enabling simultaneous assessment of central tendency and variability. Partial results are saved to Google Drive every 50 iterations.

Model configurations for the convergence experiment are deliberately lighter than in Section 4.1 to make the $100 \times 10 \times 6 = 6{,}000$ training runs feasible.

Table 11: Convergence-specific model configurations.

Model	Dataset 1	Dataset 2
Logistic Regression	`lbfgs`, C=0.3, `max_iter=10000`	`liblinear`, C=0.3, `max_iter=2000`
KNN	k=2, uniform	k=1, `kd_tree`, PCA-5
SVM	RBF, C=1, SelectKBest(k=30)	LinearSVC, C=1, `balanced`
Decision Tree	Gini, depth= 10, `min_split=20`	Gini, depth= 10, `min_split=20`
Random Forest	50 trees, depth= 10, `min_split=10`	50 trees, depth= 10, `leaf=5`
ANN	32−16−1, Adam, 10 epochs	64−32−1, Adam, 5 epochs

4.5.2 Results

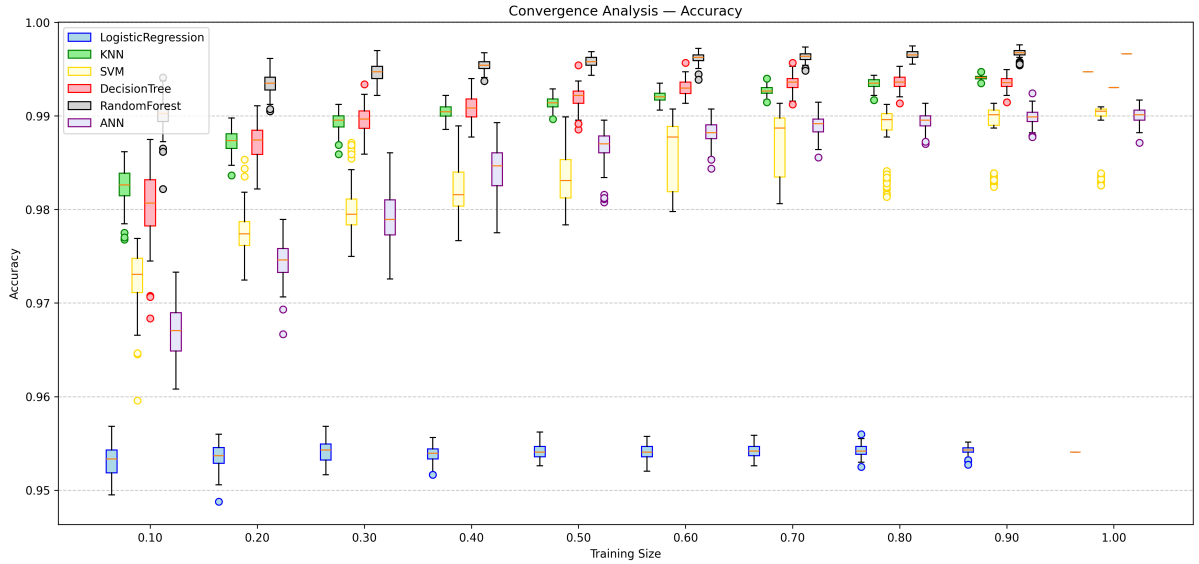

Figure 6: Convergence box plots, Accuracy, Dataset 1. Each box summarizes 100 iterations at the corresponding training fraction.

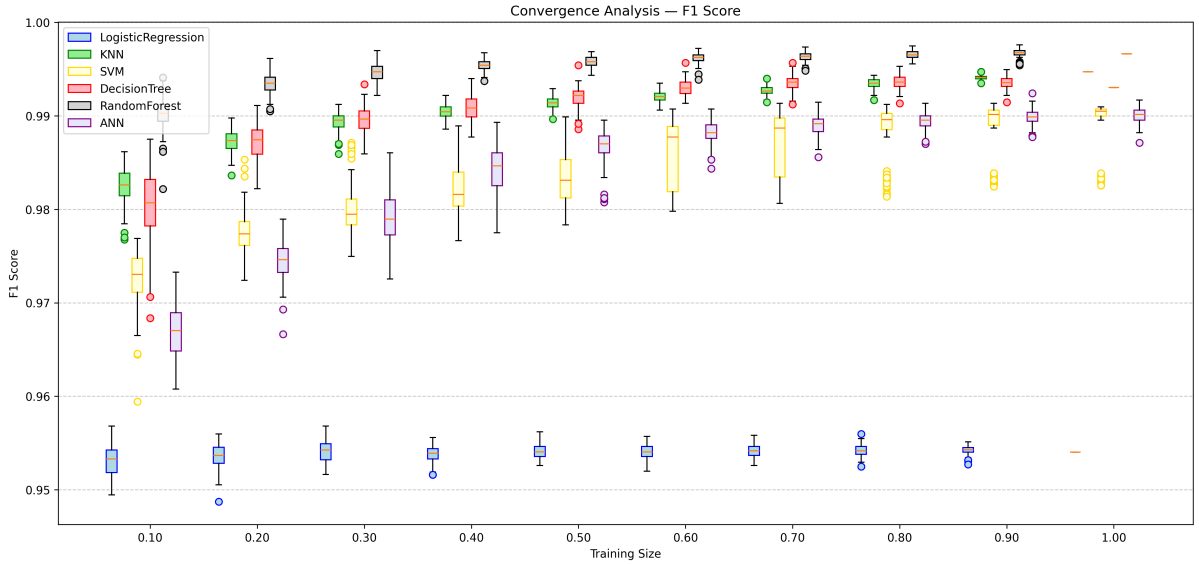

Figure 7: Convergence box plots, Weighted F1, Dataset 1.

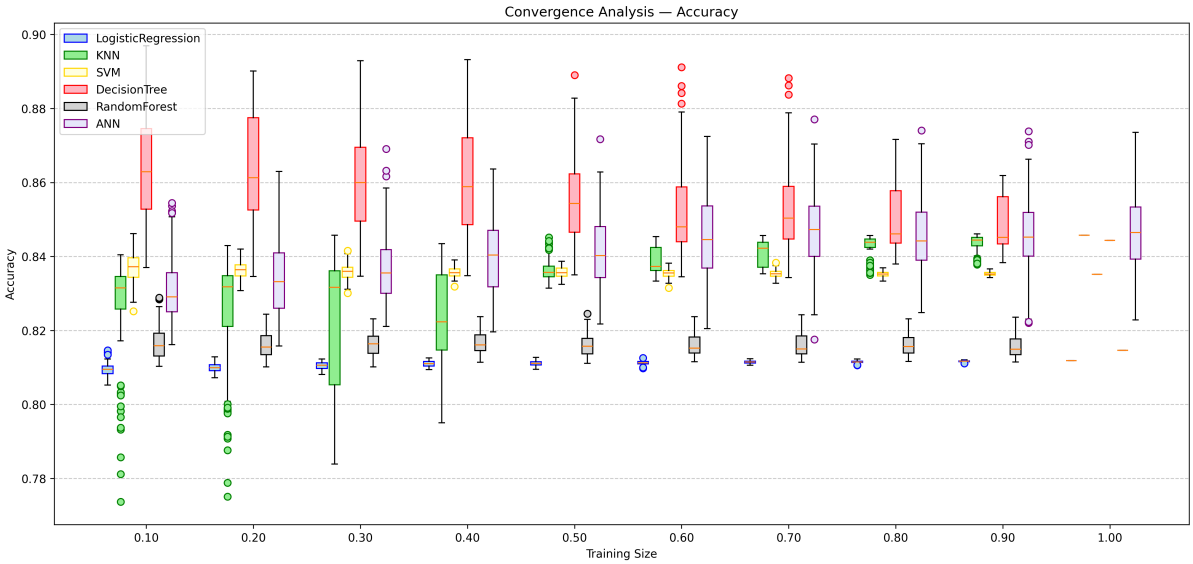

Figure 8: Convergence box plots, Accuracy, Dataset 2.

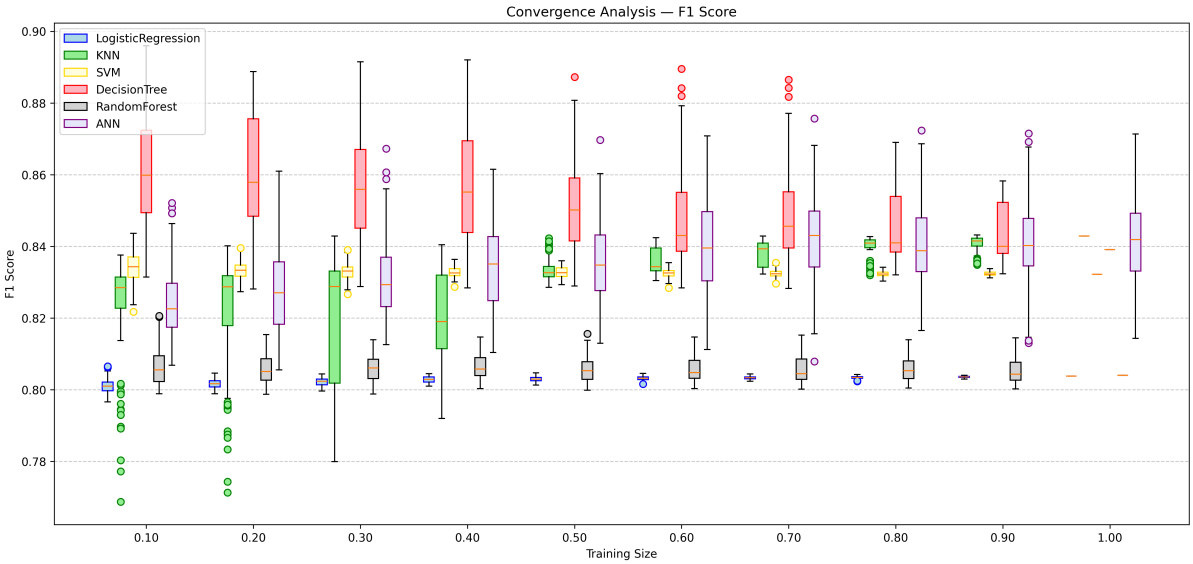

Figure 9: Convergence box plots, Weighted F1, Dataset 2.

4.5.3 Interpretation

Dataset 1. Logistic Regression converges immediately to a flat, near-zero-variance plateau across all training sizes. This confirms that its performance ceiling is set by model expressiveness (a linear boundary) rather than data quantity. Decision Tree and Random Forest reach near-optimal performance at ≈30–40% of the training data with

very low inter-iteration variance, reflecting their ability to partition the feature space efficiently even from limited samples. SVM (RBF) and ANN show slower convergence and retain residual variance even at full training size; for SVM this is due to support-vector instability under random subsampling; for ANN, to stochastic weight initialization and gradient noise within 10 epochs. KNN shows moderate improvement with training size but persistent variance due to the sensitivity of $k \in \{1, 2\}$ to local noise.

Dataset 2. Overall performance gains with training size are more modest, suggesting the bottleneck is class overlap in the feature space rather than data scarcity. Random Forest, LinearSVC, and ANN show gradual improvement and progressive reduction in variability as s increases. Decision Tree with depth= 10 fails to converge consistently, confirming insufficient capacity for the more complex UNSW-NB15 structure, in contrast to Dataset 1, where depth= 10 was adequate. Logistic Regression saturates early. KNN (PCA-5, k=1) exhibits high variance at small s, which partially decreases as more training points constrain decision regions.

Stability and overfitting. Stability is assessed by the inter-quartile range (IQR) of each box: narrow IQR at full training size indicates a stable model. Random Forest is the most stable on both datasets; KNN is the least stable. High variance at small s is indicative of overfitting to the subsample (models that memorise small training sets produce wildly different test results across iterations); this behavior is most pronounced for ANN and KNN on Dataset 2 at $s = 0.10$.

The difference in residual variance between model families has a theoretical explanation. Parametric models with convex objectives (Logistic Regression, LinearSVC) converge to a unique solution for a given training set, so their variance across iterations at fixed s reflects only sampling variability in the subsample itself. Non-parametric or stochastic models exhibit additional sources of variance: Decision Tree and Random Forest are deterministic given the data but sensitive to which samples are included, producing moderate variance at small s that vanishes as the training set grows. ANN retains residual variance even at full training size because each run uses a different random weight initialization and stochastic gradient descent trajectory, making exact convergence to the same solution unlikely within a fixed epoch budget. KNN with $k = 1$ or $k = 2$ is particularly sensitive because a single changed neighbour can flip the decision boundary, amplifying variance regardless of training size.

Table 12: Qualitative convergence summary.

Model	Dataset 1	Dataset 2
Logistic Regression	Fast, flat, low variance	Fast, flat, low variance
KNN	Moderate improvement, moderate variance	High variance, slow stabilization
SVM	Gradual improvement, residual variance	Gradual improvement, residual variance
Decision Tree	Fast, low variance (s>0.4)	Limited / inconsistent
Random Forest	Fast, near-zero variance (s>0.4)	Gradual improvement, moderate variance
ANN	Gradual improvement, residual variance	Gradual improvement, residual variance

5 Unsupervised Learning (K-Means)

5.1 Motivation and Setup

K-Means with $K = 2$ is applied to the full scaled feature matrices $\mathbf{X}_{\text{scaled}}$ of both datasets to assess whether the intrinsic geometric structure of the feature space can recover the normal/attack partition without labels. The choice $K = 2$ reflects the binary nature of the classification task. Three questions are addressed: (i) is the feature space intrinsically separable in a centroid-based sense? (ii) what discriminative advantage does supervision confer? (iii) are the two clusters geometrically coherent internally?

PCA with $n_{\text{components}} = 2$ is applied *after* clustering as a post-hoc visualization tool only; the K-Means objective is optimized in the full d-dimensional space ($d = 41$ for DS1, $d = 191$ for DS2). Mixing PCA into the clustering step would alter the Euclidean distance structure and prevent a fair assessment of cluster quality in the original feature space.

Different outcomes are expected for the two datasets. On Dataset 1, the controlled KDD simulation produces statistically distinct normal/attack profiles, so moderate-to-high external validity is expected. On Dataset 2, the nine original attack families are collapsed into a single binary label, creating an internally heterogeneous attack class that K-Means cannot cleanly partition at $K = 2$; furthermore, the high-dimensional sparse OHE space degrades Euclidean distances due to the curse of dimensionality.

5.2 PCA Visualization

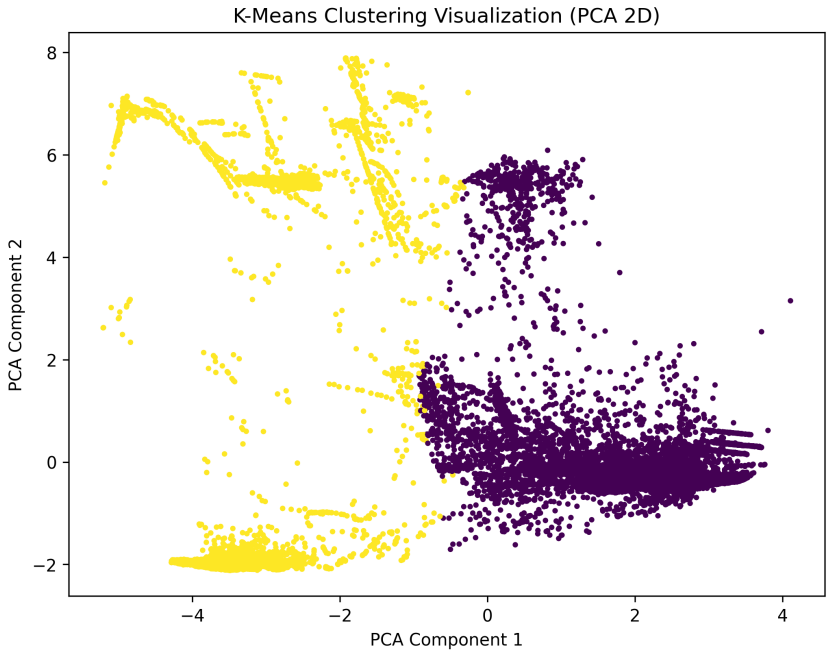

Figure 10: PCA 2D scatter plot of K-Means clusters for Dataset 1. Each point is projected onto the first two principal components of $\mathbf{X}_{\text{scaled}}$; color indicates K-Means assignment in the original 41-D space.

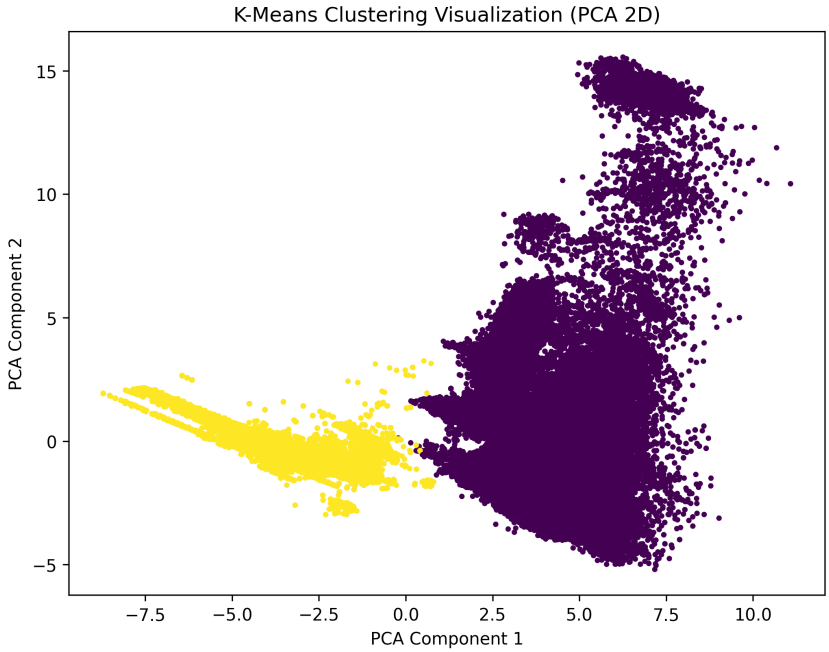

Figure 11: PCA 2D scatter plot of K-Means clusters for Dataset 2. Colour indicates cluster assignment in the full 191-D space.

For Dataset 1, the PCA projection captures only a limited portion of the variance of the original 41-D space; visual separation in the plot may therefore overstate the true degree of separability, and quantitative metrics (especially the Dunn Index) are needed for an accurate assessment. For Dataset 2, the PCA projection reveals two partially distinguishable regions, but with substantial overlap in the central area. This overlap reflects the heterogeneous nature of the attack class, which comprises nine distinct attack families sharing feature characteristics with normal traffic. The dominant principal components in the high-dimensional sparse OHE space may also capture variance driven by the `proto` encoding rather than the binary normal/attack distinction, so quantitative validity metrics are essential for an accurate interpretation.

5.3 Cluster vs. True Labels

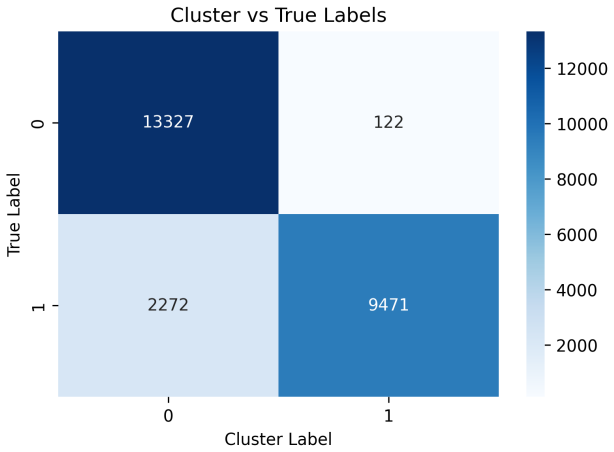

Figure 12: Heatmap of K-Means cluster assignments vs. true labels for Dataset 1. Rows: true label (0 = normal, 1 = attack); columns: cluster index.

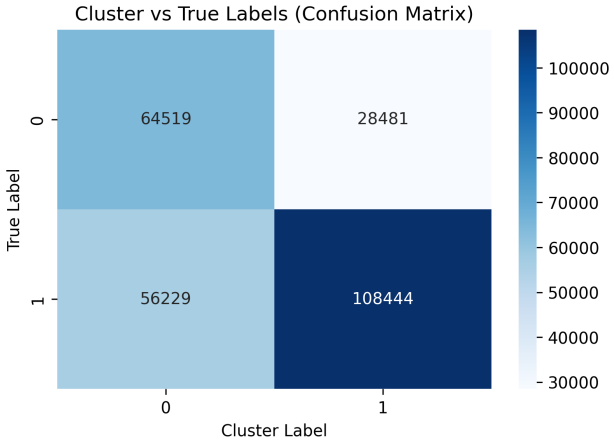

Figure 13: Heatmap of K-Means cluster assignments vs. true labels for Dataset 2. The notebook confirms only partial alignment: K-Means captures some underlying structure but the dataset is not naturally separable into the two true categories.

Table 13: Clustering validity metrics for K-Means (K=2, `random_state=42`, `n_init=10`). Internal metrics for DS2 computed on a 10 000-point subsample due to computational constraints. ↑ higher is better; ↓ lower is better.

Metric	Direction	Dataset 1	Dataset 2
External Validity			
Rand Index (RI)	↑	0.8280	0.5587
Fowlkes–Mallows Index (FMI)	↑	0.8334	0.5763
Jaccard Index (JI)	↑	0.7982	0.5614
Internal Validity			
Silhouette Index (SI)	↑	0.3507	0.1772
Davies–Bouldin Index (DBI)	↓	1.4388	2.2735
Calinski–Harabász Index (CHI)	↑	5504.37	796.83
Dunn Index (DI)	↑	0.0226	0.0764

5.4 Interpretation of Metrics

External metrics. RI measures the overall fraction of concordant pairs; a value near 0.5 corresponds to a random partition. FMI is the geometric mean of pair-precision and pair-recall, more discriminative than RI for imbalanced partitions. JI ignores correctly separated pairs (TN_c), penalizing fragmentation of ground-truth classes more strongly than RI. On Dataset 1, moderate-to-high values of all three metrics are expected, reflecting the controlled simulation environment. On Dataset 2, lower values are anticipated due to attack-class heterogeneity: K-Means will tend to fragment the nine attack sub-families into multiple sub-clusters, misaligning with the binary label.

Internal metrics. A low SI on Dataset 1, despite reasonable external scores, would indicate that the within-cluster Euclidean cohesion is not high, consistent with the notebook remark that the Dunn Index reveals considerably more overlap in the full feature space than the PCA scatter plot suggests. On Dataset 2, the high-dimensional sparse OHE space tends to flatten pairwise Euclidean distances (curse of dimensionality), degrading all internal metrics regardless of the actual cluster structure.

A notable apparent paradox in the results is that the Dunn Index for Dataset 2 (0.076) is higher than for Dataset 1 (0.023), despite all other internal metrics being worse for Dataset 2. This is not contradictory: the DI is defined as the ratio of the minimum inter-cluster distance to the maximum intra-cluster diameter, and in high-dimensional spaces the curse of dimensionality causes pairwise distances to concentrate, pushing the minimum inter-cluster distance upward even when clusters strongly overlap. The DI is therefore an unreliable metric in high-dimensional settings and should be interpreted with caution relative to SI, DBI, and CHI, which provide a more consistent picture.

Cross-dataset comparison. Dataset 1 is more naturally separable in a centroid-based sense, while Dataset 2 requires supervised discriminative power to effectively sep-

arate normal from attack traffic. This conclusion is consistent with the convergence analysis in Section 4.5, where Dataset 2 showed slower model convergence and larger benefit from additional data.

6 Project Evaluation

6.1 Critical Reflection on Methodology

Looking back at the project, several methodological choices proved effective while others would benefit from revision in a future iteration.

What worked well. The decision to treat the two datasets as parallel experiments, applying the same six models and evaluation protocol to both, proved highly informative: the consistent model ranking across datasets provides stronger evidence for the conclusions than either dataset alone would. The convergence analysis with 100 iterations per training fraction was computationally demanding but essential for reliable variance estimates; using a fixed test set throughout ensured that observed variability reflects only training-set sensitivity, not test-set noise. Saving partial results to Google Drive every 50 iterations was a practical safeguard against runtime interruptions in the Colab environment. The use of `class_weight='balanced'` on all Dataset 2 models was a deliberate and effective choice: by up-weighting the minority class during training it improved Recall for the normal class across all models, most visibly for Logistic Regression where normal recall increased from approximately 0.62 to 0.73 compared to an unweighted baseline, directly reducing the operational false-alarm rate.

What I would do differently. The most significant limitation is the *absence of systematic hyperparameter optimization for Dataset 2*. For Dataset 1, Logistic Regression was tuned via 5-fold grid search over solvers; for Dataset 2, most models use fixed configurations chosen by heuristic. A proper grid search or Bayesian optimization over the regularization strength C, tree depth, and ANN learning rate would likely improve performance, especially for Random Forest and ANN, and would make the cross-dataset comparison more balanced. Future work could explore systematic hyperparameter optimization, for instance via GridSearchCV or RandomizedSearchCV, to further improve model performance on Dataset 2. The convergence experiment deliberately used lighter configurations to keep the 6,000-run computational budget feasible, and its conclusions remain valid independently of the final model configurations.

A second consideration concerns the *K-Means setup*. Applying K-Means with $K = 2$ was the methodologically correct choice given the binary nature of the classification task: the goal was to assess whether the intrinsic geometry of the feature space reflects the normal/attack partition without supervision, and $K = 2$ is the only configuration that directly answers that question. However, a natural and informative extension would be to run a complementary experiment with $K = 10$ on the multi-class labels available in `attack_cat`, to investigate whether K-Means can recover the finer attack sub-taxonomy of UNSW-NB15. This would not replace the $K = 2$ analysis but enrich it.

Third, the *convergence models* use deliberately lighter configurations than the final models (50 trees vs. 300, 10 epochs vs. 100), a conscious tradeoff to make the $100 \times 10 \times 6 = 6{,}000$ training runs computationally feasible. The practical consequence is a systematic underestimation of the ceiling performance in the convergence curves. In a future iteration, running fewer iterations (e.g. 20–30 instead of 100) with the full final configurations would eliminate this discrepancy while remaining tractable.

6.2 Social, Ethical, and Legal Considerations

Data provenance and consent. Both datasets used in this project are publicly available benchmarks released for academic research. The KDD Cup 1999 data was collected on a simulated U.S. Air Force network; no real user traffic or personal identifiable information (PII) was captured. The UNSW-NB15 dataset was generated in a controlled hybrid testbed at UNSW Canberra using synthetic traffic and real attack tools; it does not contain PII. Their use for educational purposes raises no consent or privacy concerns.

Bias and representativeness. Both datasets are temporally dated (1999 and 2015 respectively) and were collected in specific network environments. Models trained exclusively on these benchmarks may embed *distribution shift* biases: attack patterns from 1999 (e.g. buffer overflows targeting legacy services) are largely obsolete, and UNSW-NB15, while more recent, does not reflect post-2015 attack tooling such as ransomware-as-a-service or adversarial ML attacks on IDS systems themselves. Deploying such models in a production environment without retraining on contemporary traffic would create a false sense of security.

Fairness and dual-use risk. IDS systems operate on network traffic that may include behavioral patterns correlated with demographic or organizational attributes. A high false-positive rate, a known weakness of the LinearSVC on Dataset 2, can disproportionately flag legitimate users whose traffic patterns deviate from the majority class represented in the training set. The same ML models evaluated here could in principle be repurposed for network surveillance or censorship; the techniques for distinguishing normal from attack traffic are structurally identical to those for distinguishing permitted from restricted communication. This dual-use potential warrants explicit acknowledgement when publishing or sharing IDS research.

Legal considerations. Traffic capture and storage is regulated under data protection law (e.g. GDPR in the EU); operational IDS deployments must ensure that captured packets are handled in compliance with applicable legislation. The datasets used here pre-date GDPR and were collected under the regulatory frameworks of their respective jurisdictions.

7 Conclusion

7.1 Key Findings

The five objectives stated in the introduction have been fully addressed. The dataset-specific preprocessing pipelines (objective i) demonstrated that the same raw feature space requires substantially different transformations depending on scale and heterogeneity: label encoding and a simple train/test split sufficed for Dataset 1, while Dataset 2 required cardinality reduction, Winsorization, one-hot encoding, and a combined scaler, a finding that underscores how preprocessing complexity scales with dataset realism. The evaluation of six supervised classifiers (objective ii) confirmed that non-linear models consistently outperform linear ones, with Random Forest and ANN emerging as the strongest across both benchmarks. The convergence analysis (objective iii) revealed an asymmetry that would not have been visible from a single test-set evaluation: Dataset 1 saturates early, suggesting its structure is learnable from limited data, while Dataset 2 exhibits persistent variance across all training sizes, pointing to inherent class overlap as the primary bottleneck rather than data scarcity. The K-Means clustering analysis (objective iv) provided independent geometric evidence for the same conclusion: the two datasets differ not only in size and encoding but in their fundamental intrinsic separability. Finally, the cross-dataset comparison (objective v) produced the most original contribution of this work: by holding the methodology constant and varying only the dataset, it was possible to disentangle the effect of model choice from the effect of data complexity. The result, namely that Random Forest and ANN maintain their relative advantage regardless of dataset while the ranking of all other models shifts substantially, suggests that ensemble methods and deep architectures are more robust to distribution shift than linear or instance-based classifiers, a conclusion with direct implications for practitioners who must select models for deployment in network environments that differ from their training data. Random Forest and ANN emerged as the two strongest models across all evaluations, combining high accuracy, F1-score, and ROC-AUC with good probability calibration. Crucially, this relative ranking was stable across the two datasets despite their substantial differences in origin, encoding strategy, and class distribution, a result that increases confidence in the generalizability of the conclusions.

The convergence analysis revealed an important asymmetry: models reach near-optimal performance with a smaller fraction of training data on Dataset 1 (KDD simulation) than on Dataset 2 (UNSW-NB15), where complex models continue to benefit from additional samples throughout the training-size range. This confirms that Dataset 2's bottleneck is inherent class overlap rather than data scarcity, and that more expressive models require more data to realize their full potential.

A consistent finding across all models on Dataset 2 is the difficulty in correctly identifying normal traffic: Recall for class 0 ranges from 0.59 (LinearSVC) to 0.82 (Random Forest), indicating a systematic tendency to over-predict attacks. In an operational IDS context, this translates to a high false-alarm rate that burdens security analysts. Address-

ing this imbalance, through threshold tuning, cost-sensitive learning, or more expressive feature engineering, is a concrete priority for future work.

Probability calibration, as measured by Log-Loss, varies substantially across models: Random Forest and ANN produce well-calibrated probability estimates on both datasets, while KNN (PCA-10) and Decision Tree (unbounded depth) exhibit poorly calibrated outputs, with Log-Loss values of 2.50 and 1.21 respectively on Dataset 2. In security applications where downstream decisions depend on confidence scores rather than hard labels, calibration quality is as important as discriminative performance and should be considered alongside accuracy and F1 when selecting a model for deployment.

The K-Means clustering analysis ($K = 2$) provided complementary geometric evidence: Dataset 1 is more naturally separable in a centroid-based sense, while Dataset 2 requires supervised discriminative power to effectively separate normal from attack traffic, a conclusion aligned with both the supervised performance gap and the convergence behavior.

7.2 New Questions Opened

The findings raise several directions for future investigation.

Does the model ranking hold under distribution shift? All models are trained and tested within the same dataset distribution. Cross-dataset evaluation, training on KDD and testing on UNSW-NB15 or vice versa, would reveal whether the learnt representations are portable across network environments, which is a prerequisite for real-world deployment.

How much does the binary label obscure attack structure? The UNSW-NB15 clustering results suggest that the nine attack families form geometrically distinct sub-clusters that are invisible to a $K = 2$ partition. A natural follow-up is to investigate whether supervised multi-class classification can recover this finer structure, and whether the model ranking changes when the task is attack-family identification rather than binary detection.

Can unsupervised methods detect zero-day attacks? The key practical advantage of clustering over supervised classification is that it requires no labels. However, the low external validity scores on Dataset 2 suggest that vanilla K-Means is not competitive with supervised models. More expressive density-based methods (DBSCAN) or probabilistic models (GMM) may close this gap, enabling anomaly detection for attack families absent from the training set.

What is the cost of false negatives in practice? The current evaluation treats false positives and false negatives symmetrically (accuracy, F1). In a real IDS deployment, a missed attack (false negative) is typically far more costly than a false alarm. A decision-theoretic evaluation with an asymmetric cost matrix, tuning the classification threshold to minimize expected operational cost, would provide results more directly actionable for security practitioners.

Software and Tools

All experiments were implemented in Python 3 on Google Colab (https://colab.research.google.com). The following libraries were used:

- **scikit-learn** — supervised classifiers (`LogisticRegression`, `KNeighborsClassifier`, `SVC`, `LinearSVC`, `DecisionTreeClassifier`, `RandomForestClassifier`), preprocessing (`StandardScaler`, `LabelEncoder`, `OneHotEncoder`), feature selection (`SelectKBest`, `mutual_info_classif`), dimensionality reduction (`PCA`), clustering (`KMeans`), calibration (`CalibratedClassifierCV`), and all evaluation metrics.
  https://scikit-learn.org
- **TensorFlow / Keras** — ANN implementation (`Sequential`, `Dense`, `Dropout`, `BatchNormalization`, `Adam`, `EarlyStopping`, `ReduceLROnPlateau`).
  https://www.tensorflow.org
- **pandas** — CSV loading, dataframe manipulation, train/test concatenation and split recovery.
  https://pandas.pydata.org
- **NumPy** — numerical array operations and stratified random subsampling for the convergence experiment.
  https://numpy.org
- **Matplotlib** — all figures (convergence box plots, ROC curves, PCA scatter plots, confusion matrices).
  https://matplotlib.org
- **seaborn** — heatmap visualizations of confusion matrices.
  https://seaborn.pydata.org
- **permetrics** — Dunn Index computation (`ClusteringMetric`), not available in scikit-learn.
  https://github.com/thieu1995/permetrics
- **tqdm** — progress bars for the convergence loop (6,000 training runs).
  https://tqdm.github.io
- **joblib** — model serialization and incremental checkpointing to Google Drive.
  https://joblib.readthedocs.io

The complete source code is publicly available at:

github.com/FabiFoc/Small-Project_IDS-ML

Acknowledgments

The author would like to acknowledge the use of AI-assisted tools during the development of this project. Copilot was used to support the implementation of several machine learning models and the convergence analysis pipeline, providing code suggestions that were subsequently reviewed, adapted, and integrated into the final notebook. Claude was used to assist in the writing of selected parts of this report, in particular portions of the Theoretical Background section, where it helped structure and refine technical descriptions of the algorithms. All AI-generated content, both code and text, was critically reviewed and verified by the author before inclusion.

References

[1] David Arthur and Sergei Vassilvitskii. k-means++: The advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1027–1035. Society for Industrial and Applied Mathematics, 2007.

[2] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[3] Pasi Fränti and Sami Sieranoja. How much can k-means be improved by using better initialization and repeats? *Pattern Recognition*, 93:95–112, 2019.

[4] Ansam Khraisat, Iqbal Gondal, Peter Vamplew, and Joarder Kamruzzaman. Survey of intrusion detection systems: techniques, datasets and challenges. *Cybersecurity*, 2(1):1–22, 2019.

[5] Nour Moustafa and Jill Slay. UNSW-NB15: A comprehensive data set for network intrusion detection systems. In *2015 Military Communications and Information Systems Conference (MilCIS)*, pages 1–6. IEEE, 2015.

[6] Peter J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987.

[7] Sampadab17. Network intrusion detection (kdd cup 1999). Kaggle Dataset, 2017.

[8] Mahbod Tavallaee, Ebrahim Bagheri, Wei Lu, and Ali A. Ghorbani. A detailed analysis of the KDD CUP 99 data set. In *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, pages 1–6. IEEE, 2009.